

Manipulator Modeling & Control

1 Content of this lab

The goal of this lab is to program in C++ the three fundamentals models for robot manipulators:

- Direct Geometric Model: what is the pose of the end-effector for a given joint configuration?
- Inverse Geometric Model: what joint values correspond to a given end-effector pose?
- Kinematic Model: what is the velocity screw of the end-effector when the joints move?

These models will be applied on three different robots, and used to perform basic point-to-point control.

As it is not a lab on C++, most of the code is already written and you only have to fill the required functions:

- `Robot::init_wMe()` to initialize the fixed transform between the wrist and end-effector
- `Robot::fMw(q)` for the direct geometric model wrist-to-fixed frame
- `Robot::inverseGeometry(M)` for...well, the inverse geometry
- `Robot::fJw` for the kinematic model wrist-to-fixed frame

1.1 Installing and loading the lab in Qt Creator

This project uses the ROS 2¹ framework which imposes some particular steps to configure the environment. An actual ROS course will be held in the second semester, for now just configure as follows:

1. Open a terminal and go to the ROS 2 workspace:

```
mkdir -p ~/ros2/src  
cd ~/ros2/src
```

2. Download this package:

```
git clone https://github.com/oKermorgant/ecn_manip
```

3. Compile the package with ROS 2

```
cd ecn_manip  
ros2ws  
colbuild -t
```

¹Robot Operating System, <http://www.ros.org>

4. Configure Qt Creator and load the project:

```
ideconf # or gqt on older VMs
```

5. Load the `ros2/src/ecn_manip/CMakeLists.txt` file through `File...open project`
 - You can also right-click open the file with "Qt Creator for ROS 2"
6. The files should be displayed and ready to compile and run
7. Compilation is done by clicking the bottom-left hammer
8. Run your program with the green triangle. It can be stopped by clicking on the red square

1.2 Expected work

The **only** files to be modified are:

- `control.cpp`: main file where the control is done depending on the current robot mode
- `robot_turret.cpp`: model of the RRP turret robot
- `robot_kr16.cpp`: model of the industrial Kuka KR16 robot
- `robot_ur10.cpp`: model of the Universal Robot UR-10

We will use the ViSP² library to manipulate mathematical vectors and matrices, including frame transforms, rotations, etc. The main classes are detailed in Appendix A.

2 The robots

Three robots are proposed with increasing complexity. You should start with the Turret, then the Kuka KR16 then the UR-10.

For each of these robots, the table of Modified Denavit-Hartenberg parameters should be defined. As we saw during the lectures, once you have the table then the Direct Geometric and Direct Kinematic Models can be almost automatically derived. A tool is provided in this package to generate C++ code for the Geometric and Kinematic models, see Appendix C for details on this tool.

The fixed frame and the end-effector frame are imposed by the schematics. All intermediary frames have to be placed according to MDH convention, with a final fixed transform between the wrist and end-effector frames.

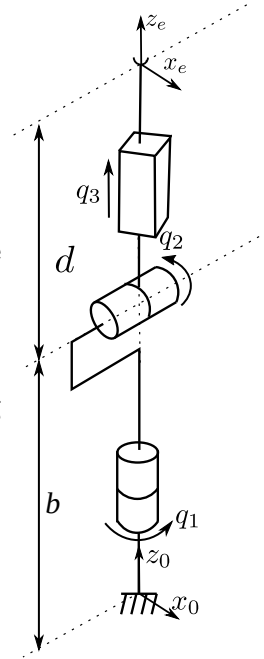
²Visual Servoing Platform, <http://visp.inria.fr>

2.1 Turret RRP robot

This is a very simple robot to begin the modeling, as shown in the figure. The fixed frame $\mathcal{F}_0$ and end-effector frame $\mathcal{F}_e$ are imposed, as well as the joint direction.

The constant values for this model are: $b = 0.5$ m and $d = 0.1$ m.

All computation may be done first without using the Denavit-Hartenberg parameters as the robot is quite simple.



2.2 Kuka KR16 anthropomorphic robot

This robot is a classical 6R architecture with a spherical wrist, as shown in the next figure:

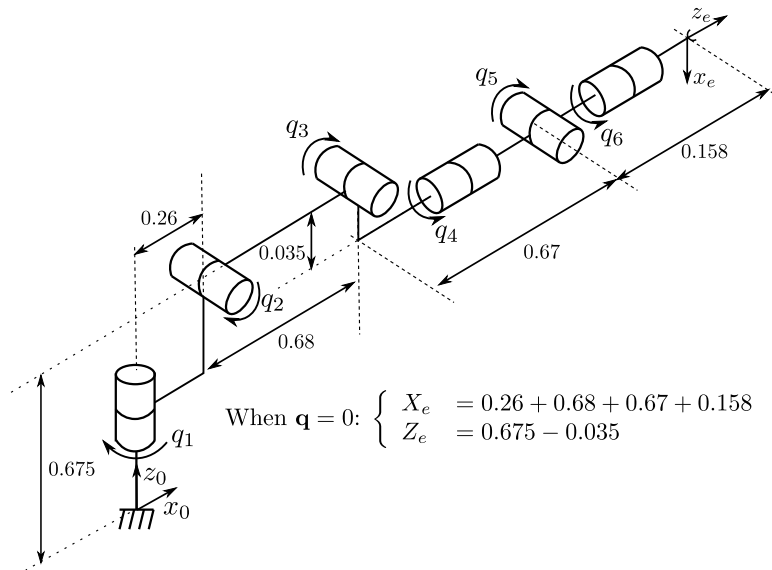


Figure 1: Model of the Kuka KR16 with imposed fixed and end-effector frames

The wrist frame is typically at the common point between joints 4, 5 and 6. The constant transform ${}^w\mathbf{M}_e$ between the wrist and the end-effector should include the 0.158 distance and potential rotations.

2.3 Universal Robot UR-10 anthropomorphic robot

This robot is a less classical 6R architecture, without spherical wrist. A tool is also attached with an arbitrary end-effector offset.

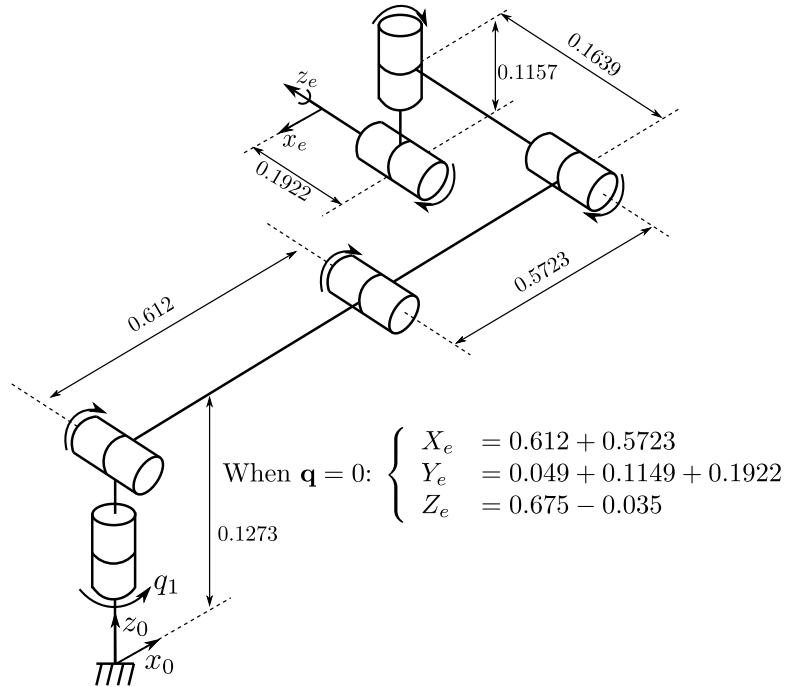


Figure 2: Model of the UR-10 with imposed fixed and end-effector frames

The Inverse Geometry is given for this robot. The constant transform ${}^w\mathbf{M}_e$ between the wrist and the end-effector should include the 0.1922 distance and potential rotations.

2.4 Running the simulation

Once everything is installed and compiled, the base simulation can be run from a terminal (only 1 line depending on the robot you work on):

```
ros2 launch ecn_manip simulation_launch.py robot:=turret
ros2 launch ecn_manip simulation_launch.py robot:=kr16
ros2 launch ecn_manip simulation_launch.py robot:=ur10
```

This will display two windows:

- RViz is the 3D simulator to show the current configuration of the robot. It will also display the base and end-effector frames, and the one that you compute. You can thus easily check if the computation is fine.
- The other window is composed of 4 panels:
 - Buttons at the top left to change the current exercise
 - Sliders at the bottom left to manually control the robot (exercises 1 and 2)
 - Sliders at the top to change the switching time of the position control (exercises 3+)
 - A plotting region with two tabs, one for the operational space and one for the joint space. The joint space is displayed in a normalized fashion, depending on the joint limits (upper and lower dotted lines)

3 Building the models

The main code

In the file `control.cpp`, the `if - else if` blocks correspond to the 6 exercises to be done for each robot. Note that if they work for the Turret robot, then they should work for all robots as long as the model is correct.

The current exercise is changed by using the buttons in the GUI.

Exercise 1: Checking the Direct Geometric Model

In this exercise, the robot follows the manually given joint positions. The task is to build and print the direct geometric model, depending on the current value of $\mathbf{q}$. This is to be done in two functions:

- `Robot::init_wMe()` where you have to write the constant matrix ${}^w\mathbf{M}_e$ from the end-effector to the wrist frame
- `Robot::fMw(q)` where you have to write and return the transform $\mathbf{M}$ between the fixed and wrist frame

Once you have computed the $\mathbf{M}$ matrix, the code calls `robot->checkPose(M)`; in order to print both your matrix and the true one. No need to tell that they should be equal.

What can be told is that it is useless to continue until they are.

You can find in Appendix B a script that can help visualizing the frames corresponding to your MDH parameters.

Exercise 2: Checking the Direct Kinematic Model (Jacobian)

If you select `Manual Operational Velocity` in the GUI then the sliders allow you to give a desired operational velocity (ie twist) for the end-effector.

This velocity is ${}^e\mathbf{v}_e$ and is defined in the end-effector frame.

The robot Jacobian has to be defined in the function `Robot::fJw(q)` that should return the Jacobian of the wrist in the fixed frame.

In the main code, the commanded velocity then to be mapped to the joint velocity in two steps:

1. Map to the fixed frame: ${}^f\mathbf{v}_e^* = \begin{bmatrix} {}^f\mathbf{R}_e & 0 \\ 0 & {}^f\mathbf{R}_e \end{bmatrix} {}^e\mathbf{v}_e^*$

Call `M.getRotationMatrix()` to get this rotation.

The `ecn::putAt` function may be of good use.

2. Map to the joint space: $\dot{\mathbf{q}} = \mathbf{J}^+ \cdot {}^f\mathbf{v}_e^*$

Call `robot->fJe(q).pseudoInverse()` to get the Jacobian pseudo-inverse

You can check that the end-effector is able to follow straight 3D lines in x, y or z direction in its own frame. Similarly, it should be able to rotate around x, y or z. This means the Jacobian computation is correct.

Exercise 3: Checking the Inverse Geometric Model

If you select `Direct point-to-point` in the GUI then the robot will receive two operational setpoints, at a given sampling time that can be changed with the `Switching time` slider in the GUI. The current setpoint is in the matrix `ma` and its corresponding joint position is in the vector `qf`. Nothing has to be done in the main code, but of course the `Robot::inverseGeometry(M)` function has to be filled in order to compute the actual inverse geometry solution. The robot will try to have all joints reach their desired value as soon as possible, which will lead to a discontinuous motion.

In practice, this exercise is to check that the Inverse Geometric Model is correct: the robot should reach the desired pose.

When computing the inverse geometry, try to find a solution as near as possible to the passed `q0`.

Many useful functions are available to solve classical trigonometric equations. They are detailed in Appendix D.

A given potential solution should be stored using `addCandidate({q1, q2, ..., qn});`

At the end of the Inverse Geometry function, use `return bestCandidate(q0);` that will pick the candidate that is the nearest from the current joint position and also within joint limits.

Exercise 4: Interpolated point-to-point control

In this exercise the previous pose setpoint `m0` and its corresponding joint position `q0` should be used to interpolate the joint positions.

Here the goal is to compute the current setpoint $\mathbf{q}_c = \mathbf{q}_0 + p(t)(\mathbf{q}_f - \mathbf{q}_0)$. The interpolation function $p(t)$ was defined during the lectures and should take into account the maximum joint velocity and acceleration that are stored in the `vMax` and `aMax` vectors. Basically this amounts to computing the minimum time `tf` needed to reach `qf` from `q0`. This computation should be done in the `if(robot->newRef())` and then used to compute the current `qCommand`.

Exercise 5: Operational control through Inverse Geometric Model

Exercises 3 and 4 lead to the correct pose but not in a straight 3D line. Indeed the robot is only controlled in the joint space with no constraints between the two extreme poses.

In this exercise, the goal is to perform the interpolation not in the joint space but in the Cartesian space. In practice, we will compute many poses between `m0` and `ma` and command the robot to reach sequentially all these poses. As they will be pretty close one from each other, the resulting motion will be close to a straight line.

The `robot->intermediaryPose(M0, Md, alpha)` function returns a pose between `m0` and `ma`, where `alpha` is between 0 and 1 and should be computed from t , t_0 and $t_f = 1$ s.

Then, the inverse geometry of this pose should be sent to the robot as joint position setpoint.

Exercise 6: Operational control through Jacobian

In this exercise the goal is still to follow a straight line between the two points, but this time by sending a joint velocity command.

As seen in class, this command should make the end-effector approach its desired pose. The steps are:

1. Compute the pose error in the desired frame: ${}^{e*}\mathbf{M}_e = {}^f\mathbf{M}_{e*}^{-1} \cdot {}^f\mathbf{M}_e$
 - in the code, ${}^f\mathbf{M}_{e*}$ corresponds to `m_d` and ${}^f\mathbf{M}_e$ to `m`
 - In practice we use the $(\mathbf{t}, \theta\mathbf{u})$ representation with `p.buildFrom()`
2. Compute the desired linear and angular velocities: $v = -\lambda {}^f\mathbf{M}_{e*}\mathbf{t}$, $\omega = -\lambda {}^f\mathbf{M}_e(\theta\mathbf{u})$
3. Map these velocities to the joint space: $\dot{\mathbf{q}} = \mathbf{J}^+ \begin{bmatrix} v \\ \omega \end{bmatrix}$

λ is a gain that can be changed from the GUI. Increasing it will lead to a faster convergence. It can be obtained through `robot->lambda()`.

A Using ViSP

ViSP is a library for Visual Servoing (wait for the M2 Robotics to know more!) and also includes all necessary tools to play with mathematical vectors, matrices and 3D transforms. The full documentation is here: <http://visp-doc.inria.fr/doxygen/visp-3.1.0/classes.html>.

The main classes to be used are:

- `vpMatrix`: a classical matrix of given dimensions, can be multiplied with other matrices and vectors if the dimensions are ok. The pseudo-inverse of matrix `m` is available with `M.pseudoInverse()`
- `vpColVector`: a column vector of given dimension
- `vpHomogeneousMatrix`: the classical 4×4 3D transform matrix

Most of the model computation is about initializing a matrix or vector to correct dimensions (which is already done) and fill its elements with formulae.

To write 1 at element (i,j) of matrix M, just write `M[i][j] = 1;`

Similarly, to write 1 at element (i) of vector q, just write `q[i] = 1;`

3D transforms can be easily changed between homogeneous matrix, rotation matrix, translation vector or $\theta\mathbf{u}$ vector. Another type called a pose vector consists in the full $(\mathbf{t}, \theta\mathbf{u})$ vector of dimension 6:

```
vpHomogeneousMatrix M;
vpPoseVector p;
vpRotationMatrix R;
vpTranslationVector t;
vpColVector tu(3);

M.getRotationMatrix()*t; // extract the rotation part from M and multiply with t
p.buildFrom(M); // build (t, theta-u) from M
t = M.getTranslationVector(); // extract the translation part

// copy the theta-u part of p into tu
for(int i = 0; i < 3; ++i)
    tu[i] = p[i+3];

// compute inverse transform
M = M.inverse();
```

Two utility functions are provided to put a matrix or vector into a bigger one:

- `void ecn::putAt(M, Ms, r, c)`: Writes matrix `Ms` inside matrix `M`, starting from given row and column.
- `void ecn::putAt(V, Vs, r)`: Writes column vector `Vs` inside vector `V`, starting from given row. These two functions do nothing but print an error if the submatrix or vector does not fit in the main one.

B Checking the frames from MDH

A script in this package can load your MDH parameter file and generate the resulting robot to check if it corresponds to your frames. A section `constants` should be added in order to get the numerical values:

```
notation: [alpha, a, theta, r]
joint:
  1: [0, 0, q1, 0]
  2: [-pi/2, 0, 0, q2]
  3: [pi/2, 0, q3, r3]
  4: [-pi/2, a4, q4+pi/2, 0]
constants:
  r3: 0.1
  a4: 0.7
```

Then just run:

```
ros2 launch ecn_manip mdh_launch.py mdh:=<your file.yaml>
```

This can be useful when testing your direct geometric model.

C Code generation from Modified Denavit-Hartenberg parameters

The core of robot modeling is to build the MDH table. From this, the DGM and KM can be derived automatically. The Inverse Geometry still has to be done by hand (even if some advanced tools allow code-generation for the IG resolution).

As it is quite tedious to have to compute all matrices when a solution of MDH is investigated, a tool is provided to generate the C++ code. An example can be found in the file `dh_example.yml`:

```
notation: [alpha, a, theta, r]
joint:
  1: [0, 0, q1, 0]
  2: [-pi/2, 0, 0, q2]
  3: [pi/2, 0, q3, r3]
  4: [-pi/2, a4, q4+pi/2, 0]
```

The command to be run is: `roslaunch ecn_manip dh_code.py <name of the file>`.

In the case of the above MDH table, it leads to these outputs:

```
// Generated pose code
const double c1 = cos(q[0]);
const double c1p3 = cos(q[0]+q[2]);
const double c4 = cos(q[3]);
const double s1 = sin(q[0]);
const double s1p3 = sin(q[0]+q[2]);
const double s4 = sin(q[3]);
M[0][0] = -s4*c1p3;
M[0][1] = -c4*c1p3;
M[0][2] = -s1p3;
M[0][3] = a4*c1p3 - q[1]*s1;
M[1][0] = -s4*s1p3;
M[1][1] = -s1p3*c4;
M[1][2] = c1p3;
M[1][3] = a4*s1p3 + q[1]*c1;
M[2][0] = -c4;
M[2][1] = s4;
M[2][2] = 0;
M[2][3] = r3;
M[3][0] = 0;
M[3][1] = 0;
M[3][2] = 0;
M[3][3] = 1.;
// End of pose code
```

```
// Generated Jacobian code
const double c1 = cos(q[0]);
const double c1p3 = cos(q[0]+q[2]);
const double s1 = sin(q[0]);
const double s1p3 = sin(q[0]+q[2]);
J[0][0] = -a4*s1p3 - q[1]*c1;
J[0][1] = -s1;
J[0][2] = -a4*s1p3;
//J[0][3] = 0;
J[1][0] = a4*c1p3 - q[1]*s1;
J[1][1] = c1;
J[1][2] = a4*c1p3;
//J[1][3] = 0;
//J[2][0] = 0;
//J[2][1] = 0;
//J[2][2] = 0;
//J[2][3] = 0;
//J[3][0] = 0;
//J[3][1] = 0;
//J[3][2] = 0;
J[3][3] = -s1p3;
//J[4][0] = 0;
//J[4][1] = 0;
//J[4][2] = 0;
J[4][3] = c1p3;
J[5][0] = 1.;
//J[5][1] = 0;
J[5][2] = 1.;
//J[5][3] = 0;
// End of Jacobian code
```

This is exactly what you would get by doing the manual computation from the initial table, which I hope is quite appreciated.

Fixed transforms for initial and end-effector frames can also be written with keys `f:` and `e:`:

If passing `--display` the script will display the direct geometry with classical math notations. It can be useful for the inverse geometry computation.

For a 6-DOF robot, passing `--wrist` will display useful information to benefit from a spherical wrist: math expressions for ${}^0\mathbf{t}_6(q_1, q_2, q_3)$ and ${}^3\mathbf{R}_6(q_4, q_5, q_6)$, and code for ${}^0\mathbf{R}_3(q_1, q_2, q_3)$.

D Trigonometric solvers

The code contains solvers for all types of trigonometric equations (except type 1 that is obvious). They take as argument the coefficients of the equations. They return:

- A `std::vector<double>` for equations of 1 unknown (types 2 & 3).

We can go through the valid solutions with:

```
// assume q1 should ensure type 2: X.sin(q1) + Y.cos(q1) = Z

for(double q1: solveType2(X, Y, Z))
{
    // here q1 is a valid solution, may be used to deduce other joint values
}
```

- A `std::vector<JointSolution>` for equations of 2 unknowns (types 4 to 8).

In this case, the valid solutions can be explored with:

```
// assume q1 and q2 should ensure type 6 equation:
// W.sin(q2) = X.cos(q1) + Y.sin(q1) - Z1
// W.cos(q2) = X.sin(q1) - Y.cos(q1) - Z1

for(auto qs: solveType6(X, Y, Z1, Z2, W))
{
    // here a (q1,q2) solution is available in qs.qi and qs.qj
}
```

Furthermore, a function allows retrieving the 12 elements of ${}^0\mathbf{M}_{n*}$ from the matrix ${}^f\mathbf{M}_{e*}$ given to the inverse geometry solver:

```
const auto [xx,xy,xz,yx,yy,yz,zx,zy,zz,tx,ty,tz] = explodeMatrix(fMe_des);
```

This function output the matrix terms according to:

$${}^0\mathbf{M}_{n*} = {}^0\mathbf{M}_f {}^f\mathbf{M}_{e*} {}^e\mathbf{M}_n = \begin{bmatrix} x_x & y_x & z_x & t_x \\ x_y & y_y & z_y & t_y \\ x_z & y_z & z_z & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Similarly, when solving a spherical wrist, once the ${}^0\mathbf{R}_3$ matrix was computed then the ${}^3\mathbf{R}_6$ can be decomposed with:

```
const auto [xx,xy,xz,yx,yy,yz,zx,zy,zz] = explodeWristMatrix(fMe_des, R03);
```

This function output the matrix terms according to:

$${}^3\mathbf{R}_6 = {}^0\mathbf{R}_f {}^f\mathbf{R}_{e*} {}^e\mathbf{R}_n = \begin{bmatrix} x_x & y_x & z_x \\ x_y & y_y & z_y \\ x_z & y_z & z_z \end{bmatrix}$$

Note that the $(x_x, \dots, z_z)$ terms are not the same as when decomposing the full ${}^0\mathbf{M}_{n*}$ matrix.

Below are listed the syntax for all trigonometric solvers:

Type	Equation	Syntax	Returns (vector of)
1	$Xq_i = Y$	does not exist	q_i
2	$Xs_i + Yc_i = Z$	<code>solveType2(x, y, z)</code>	q_i
3	$X_1s_i + Y_1c_i = Z_1$ $X_2s_i + Y_2c_i = Z_2$	<code>solveType3(x1, y1, z1, x2, y2, z2)</code>	q_i
4	$X_1q_js_i = Y_1$ $X_2q_jc_i = Y_2$	<code>solveType4(x1, y1, x2, y2)</code>	(q_i, q_j)
5	$X_1s_i = Y_1 + Z_1q_j$ $X_2c_i = Y_2 + Z_2q_j$	<code>solveType5(x1, y1, z1, x2, y2, z2)</code>	(q_i, q_j)
6	$Ws_j = Xc_i + Ys_i + Z_1$ $Wc_j = Xs_i - Yc_i + Z_2$	<code>solveType6(x, y, z1, z2, w)</code>	(q_i, q_j)
7	$W_1c_j + W_2s_j = Xc_i + Ys_i + Z_1$ $W_1s_j - W_2c_j = Xs_i - Yc_i + Z_2$	<code>solveType7(x, y, z1, z2, w1, w2)</code>	(q_i, q_j)
8	$Xc_i + Yc_{ij} = Z_1$ $Xs_i + Ys_{ij} = Z_2$	<code>solveType8(x, y, z1, z2)</code>	(q_i, q_j)